

Test Design Proposal for Price Filter Functionality - Raxmon Mamadiyrov

Test Design Techniques

1. **Equivalence Partitioning:**
 - Test with prices that are partly correct and partly wrong to make sure the system works well.
 - For example: Use prices that are inside and outside the normal product price range (like min: \$10, max: \$100).
 2. **Boundary Value Analysis:**
 - Focus on the edges of price ranges to confirm correct handling.
 - **Example:** Testing at the boundaries, like minimum = \$0, \$1, maximum = \$99, \$100.
 3. **Error Guessing:**
 - Use intuition to identify potential failure points, such as entering invalid characters in the price fields.
 - **Example:** Testing input of negative numbers, non-numeric characters, and extreme values.
 4. **State Transition Testing:**
 - Verify how the filter transitions between states when a user adjusts the price range or resets it.
 - **Example:** Changing from a filtered state back to the default view after resetting filters.
 5. **Usability Testing:**
 - Evaluate how intuitive and user-friendly the price filter interface is.
 - **Example:** Testing for clear labels and accessible reset functionality.
-

Test Cases

Test Case 1: Setting Valid Price Range

- **Test Summary:** Verify that the system updates search results when a valid price range is applied.
- **Preconditions:** The product database is populated with prices ranging from \$1 to \$1000.
- **Steps:**
 1. Navigate to the search page.
 2. Set a minimum price of \$10 and a maximum price of \$100.
 3. Observe the search results.
- **Expected Result:** Only products within the \$10 to \$100 range are displayed.
- **Testing Type:** Functional
- **Priority:** High
- **Additional Attributes:**
 - Device: Desktop, Mobile
 - Browser: Chrome, Firefox

Test Case 2: Handling Empty Price Fields

- **Test Summary:** Ensure the system displays all products when the price fields are left empty.
- **Preconditions:** None.
- **Steps:**
 1. Leave the minimum and maximum price fields blank.
 2. Observe the search results.
- **Expected Result:** All products are displayed without filtering.
- **Testing Type:** Functional
- **Priority:** Medium
- **Additional Attributes:**
 - Accessibility: Tested with screen readers

Test Case 3: Boundary Value Validation

- **Test Summary:** Confirm the filter handles boundary values correctly.
- **Preconditions:** Prices range from \$1 to \$1000.

- **Steps:**
 1. Set the minimum price to \$1 and maximum to \$1000.
 2. Observe the search results.
- **Expected Result:** All products from \$1 to \$1000 are displayed.
- **Testing Type:** Boundary
- **Priority:** High
- **Additional Attributes:**
 - Device: Mobile

Test Case 4: Resetting Price Filters

- **Test Summary:** Verify the reset functionality returns all results.
- **Preconditions:** A price filter is applied.
- **Steps:**
 1. Apply a price range filter (e.g., \$20 to \$50).
 2. Click the "Reset Filters" button.
 3. Observe the search results.
- **Expected Result:** All products are displayed, with no filters applied.
- **Testing Type:** Functional
- **Priority:** Medium
- **Additional Attributes:**
 - Compatibility: Tested on multiple devices

Test Case 5: Invalid Input Handling

- **Test Summary:** Confirm the system handles invalid inputs gracefully.
- **Preconditions:** None.
- **Steps:**
 1. Enter letters or special characters in the price fields.
 2. Observe the system behavior.
- **Expected Result:** The system shows an error message or prevents input.
- **Testing Type:** Negative
- **Priority:** High
- **Additional Attributes:**
 - Localization: Tested in multiple languages

Test Execution and Results:

Platform: <https://www.amazon.com/>

- **Test Case 1: Passed.**
- **Test Case 2: Passed.**
- **Test Case 3: Passed.**
- **Test Case 4: Passed.**
- **Test Case 5: Failed.**
 - **Bug Found:** When entering special characters (e.g., @, #) in the price fields, the system displayed a blank search result without showing an error message.

Bug Report

- **Title:** Invalid input in price filter fields causes blank results.
- **Steps to Reproduce:**
 1. Navigate to the search page.
 2. Enter special characters in the price filter fields.
 3. Apply the filter.
- **Expected Result:** Error message is displayed.

- **Actual Result:** Blank search result is shown.
 - **Severity:** Medium
 - **Priority:** High
-

Ad Hoc Checklist

1. **Verify the price filter functionality on different browsers (Chrome, Safari, Firefox, Edge).**
2. **Test the responsiveness of the filter UI on desktop, tablet, and mobile devices.**
3. **Ensure that filters persist after a page refresh.**
4. **Check compatibility with voice-controlled input (e.g., screen readers).**
5. **Validate that applying multiple filters (e.g., price + category) works correctly.**
6. **Test performance with large datasets (e.g., 10,000+ products).**
7. **Ensure the price filter updates results dynamically without requiring a page reload.**
8. **Test for accurate currency conversion when different currencies are used.**

Additional Comments

- Suggest improving error handling for invalid inputs to ensure a clear user experience.
- Recommend automated testing to verify dynamic updates and responsiveness across devices.
- Include user feedback loops for usability improvements.